

Web Technology and Java Programming

Complete Semester Exam Notes — Units 1 to 5

Contents

Unit 1: Internet, HTML/DHTML, and Core Java Fundamentals	3
Part A: Internet and HTML	3
Part B: Core Java Fundamentals	8
Unit 2: Core Java — Exception Handling, Threads, I/O, Applet, AWT, JDBC	13
2.1 Exception Handling	13
2.2 Multithreaded Programming	13
2.3 I/O (Input/Output) in Java	14
2.4 Applet	14
2.5 String Handling	15
2.6 Networking	15
2.7 Event Handling	16
2.8 Introduction to AWT (Abstract Window Toolkit)	16
2.9 AWT Controls (Components)	16
2.10 Layout Managers	17
2.11 Menus	17
2.12 Images and Graphics	18
2.13 JDBC (Java Database Connectivity)	18
Unit 3: Java Servlets and JSP	19
Part A: Java Servlets	19
Part B: JSP (JavaServer Pages)	21
Unit 4: Java Swing	25
4.1 Introduction to Swing	25
4.2 Creating a Swing Applet and Application	25
4.3 Programming Using Panes	25
4.4 Pluggable Look and Feel (PLAF)	26
4.5 Basic Swing Controls	26
4.6 Viewport, Scroll Panes, and Scroll Bars	27
4.7 Lists and Combo Box	27
4.8 Progress Bar	27
4.9 Menus and Toolbars	27
4.10 Layered Panes	28
4.11 Tabbed Panes	28

4.12 Split Panes	28
4.13 Layouts (Swing Layout Managers)	28
4.14 Windows and Dialog Boxes	28
4.15 Internal Frame	29
Unit 5: Java Beans, Enterprise Java Beans (EJB), and RMI	30
5.1 Introduction to Java Beans	30
5.2 Application Builder Tools	30
5.3 The Bean Developer Kit (BDK)	30
5.4 JAR Files	30
5.5 Introspection	31
5.6 Developing a Simple Bean	31
5.7 Using Bound Properties	31
5.8 The Java Bean API	32
5.9 Session Beans (EJB)	32
5.10 Entity Beans (EJB)	33
5.11 Introduction to Enterprise Java Beans (EJB)	33
5.12 Introduction to RMI (Remote Method Invocation)	33

Unit 1: Internet, HTML/DHTML, and Core Java Fundamentals

Part A: Internet and HTML

1.1 Introduction to Internet

The **Internet** is a global network of interconnected computer networks that communicate using a standard set of protocols, primarily **TCP/IP (Transmission Control Protocol/Internet Protocol)**.

Key characteristics: decentralized, packet-switched, connects millions of devices worldwide, enables information sharing and communication.

Important Terminologies: | Term | Description | |—|—| | **WWW (World Wide Web)** | A system of interlinked hypertext documents accessed via the Internet using browsers | | **URL (Uniform Resource Locator)** | Address used to locate a resource on the web, e.g., <https://www.example.com/page.html> | | **HTTP/HTTPS** | Protocol used to transfer web pages (HTTPS = secure, encrypted version) | | **ISP (Internet Service Provider)** | Company that provides Internet access | | **IP Address** | Unique numeric address identifying a device on a network | | **DNS (Domain Name System)** | Translates human-readable domain names into IP addresses | | **Web Browser** | Software used to access and display web pages (Chrome, Firefox) | | **Web Server** | Computer/software that stores and delivers web pages to clients on request |

1.2 Internet Services

Service	Description
WWW	Hypertext-based information retrieval system
E-mail	Electronic mail service for sending/receiving messages
FTP (File Transfer Protocol)	Used to transfer files between computers over a network
Telnet	Allows remote login to another computer over the network
Chat/Instant Messaging	Real-time text communication
VoIP (Voice over IP)	Voice communication over the Internet
Video Conferencing	Real-time audio-video communication between multiple users
Search Engines	Tools (Google, Bing) to search for information on the web
Social Networking	Services enabling users to connect and share content (Facebook, LinkedIn)
Cloud Computing Services	On-demand delivery of computing resources over the Internet

1.3 HTML (HyperText Markup Language) — Overview

HTML is a **markup language** used to create and structure content on the web using **tags** (elements enclosed in angle brackets < >). Most tags come in pairs: an opening tag <tag> and a closing tag </tag>.

Basic Structure of an HTML document:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <!-- Page content goes here -->
  </body>
</html>
```

1.4 Formatting Tags

Tag	Purpose
 / 	Bold text
<i> / 	Italic text
<u>	Underlined text
<mark>	Highlighted text
<small>	Smaller text
<sub> / <sup>	Subscript / Superscript
<h1> to <h6>	Headings (h1 largest, h6 smallest)
<p>	Paragraph
 	Line break (empty tag)
<hr>	Horizontal rule/line
<pre>	Preformatted text (preserves whitespace)
<blockquote>	Indented quotation block
	Specifies font, size, color (deprecated — CSS preferred)

Example:

```
<h1>Main Heading</h1>
<p>This is a <b>bold</b> and <i>italic</i> paragraph.</p>
<hr>
```

1.5 Links (Hyperlinks)

Created using the **<a> (anchor) tag** with the href attribute specifying the destination.

```
<a href="https://www.example.com">Visit Example</a>
<a href="page2.html">Go to Page 2</a>
<a href="#section1">Jump to Section 1</a>      <!-- internal link -->
```

```
<a href="mailto:user@mail.com">Send Email</a>      <!-- email link -->
<a href="page.html" target="_blank">Open in new tab</a>
```

1.6 Lists

Unordered List (bullets) — with items:

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
</ul>
```

Ordered List (numbered) — with items:

```
<ol type="1">
  <li>Step One</li>
  <li>Step Two</li>
</ol>
```

Definition List — <dl>, <dt> (term), <dd> (description):

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
</dl>
```

1.7 Tables

Created using <table>, with <tr> (table row), <th> (table header cell), <td> (table data cell).

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Marks</th>
  </tr>
  <tr>
    <td>Ravi</td>
    <td>85</td>
  </tr>
</table>
```

Attributes: border, cellpadding, cellspacing, rowspan (merge rows), colspan (merge columns), width.

1.8 Frames

Frames allow dividing a browser window into multiple independent sections, each displaying a separate HTML document. (*Note: <frame>/<frameset> are deprecated in HTML5, replaced largely by <i>iframe</i>.*)

```
<frameset cols="25%,75%">
  <frame src="menu.html">
  <frame src="content.html">
</frameset>
```

<iframe> (Inline Frame) — embeds another HTML document within the current page:

```
<iframe src="page.html" width="300" height="200"></iframe>
```

1.9 Forms

Used to collect user input, created with `<form>` and various input controls.

```
<form action="submit.php" method="post">
  Name: <input type="text" name="uname"><br>
  Password: <input type="password" name="pwd"><br>
  Gender: <input type="radio" name="gender" value="M">Male
         <input type="radio" name="gender" value="F">Female<br>
  <input type="checkbox" name="sub" value="Math">Math<br>
  <select name="city">
    <option value="delhi">Delhi</option>
    <option value="mumbai">Mumbai</option>
  </select><br>
  <textarea name="comments" rows="4" cols="30"></textarea><br>
  <input type="submit" value="Submit">
  <input type="reset" value="Reset">
</form>
```

Key attributes: action (URL to send data to), method (GET or POST), name, type (text, password, radio, checkbox, submit, reset, file, hidden, email, date, etc.).

1.10 Comments in HTML

Comments are not displayed by the browser; used for documentation/notes in the source code.

```
<!-- This is a comment and will not be displayed -->
```

1.11 DHTML (Dynamic HTML)

DHTML is **not a separate language** but a **combination of technologies** — **HTML + CSS + JavaScript (and DOM)** — used together to create interactive and dynamic web pages that can change content, style, and structure **without reloading the page**.

Key components: | Component | Role | |—|—| | **HTML** | Provides the basic structure/content | | **CSS** | Controls the presentation/style (can be changed dynamically) | | **JavaScript** | Adds behavior/interactivity, manipulates content dynamically | | **DOM**

(Document Object Model) | Tree-structure representation of the HTML page that JavaScript uses to access and modify elements |

Features enabled by DHTML: animations on the page, dynamic content updates, drag-and-drop, dynamic positioning of elements, form validation, interactive menus — all handled at the client side without server requests.

Example (simple DHTML interactivity):

```
<p id="demo">Hello!</p>
<button onclick="document.getElementById('demo').innerHTML='Changed!'">
  Click Me
</button>
```

Part B: Core Java Fundamentals

1.12 Introduction to Java

Java is a **high-level, object-oriented, platform-independent** programming language developed by **Sun Microsystems** (James Gosling), released in 1995, now owned by Oracle.

Key Features: | Feature | Meaning | |—|—| | **Platform Independent** | “Write Once, Run Anywhere” — compiled bytecode runs on any device with a JVM | | **Object-Oriented** | Everything is organized around objects and classes | | **Simple** | Syntax is cleaner than C++ (no pointers, no operator overloading) | | **Secure** | No explicit pointers, runs in a sandboxed JVM environment | | **Robust** | Strong memory management, exception handling, automatic garbage collection | | **Multithreaded** | Built-in support for concurrent execution of multiple threads | | **Architecture Neutral / Portable** | Bytecode format is independent of underlying hardware | | **Distributed** | Supports networking and distributed applications (RMI, sockets) | | **Interpreted & Compiled** | Source code compiled to bytecode (.class), interpreted by JVM at runtime |

JVM, JRE, JDK: - **JVM (Java Virtual Machine):** executes Java bytecode; provides platform independence. - **JRE (Java Runtime Environment):** JVM + libraries needed to *run* Java programs. - **JDK (Java Development Kit):** JRE + development tools (compiler `javac`, debugger) needed to *develop* Java programs.

Basic Java Program:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

1.13 Operators in Java

Category	Operators
Arithmetic	+ - * / %
Relational	== != > < >= <=
Logical	&& \ \ !
Assignment	= += -= *= /= %=
Increment/Decrement	++ -- (pre and post)
Bitwise	& \ ^ ~ << >> >>>
Conditional (Ternary)	condition ? expr1 : expr2
instanceof	Tests if an object is an instance of a class

Example: `int max = (a > b) ? a : b;`

1.14 Data Types in Java

Java is **strongly (statically) typed**. Data types are divided into two categories:

Primitive Data Types (8 types): | Type | Size | Range/Description | |—|—|—| | byte | 1 byte | -128 to 127 | | short | 2 bytes | -32,768 to 32,767 | | int | 4 bytes | -2³¹ to 2³¹-1 | | long | 8 bytes | -2⁶³ to 2⁶³-1 | | float | 4 bytes | Single-precision decimal | | double | 8 bytes | Double-precision decimal | | char | 2 bytes | Single 16-bit Unicode character | | boolean | 1 bit (JVM-dependent) | true or false |

Non-Primitive (Reference) Data Types: Class, Array, String, Interface — store references (addresses) to objects, not the actual value.

1.15 Variables

A **variable** is a named memory location used to store data that can change during program execution.

Types of variables: - **Local variable:** declared inside a method; exists only during that method's execution. - **Instance variable:** declared inside a class but outside methods; each object has its own copy. - **Static (class) variable:** declared with static keyword; shared among all objects of the class (one copy per class).

```
class Demo {
    int instanceVar = 10;           // instance variable
    static int staticVar = 20;     // static variable
    void method() {
        int localVar = 30;        // local variable
    }
}
```

1.16 Arrays

An **array** is a fixed-size, ordered collection of elements of the **same data type**, stored in contiguous memory, accessed via an index (starting from 0).

Declaration & Initialization:

```
int[] arr = new int[5];           // 1D array, default values = 0
int[] nums = {10, 20, 30, 40};    // direct initialization
int[][] matrix = new int[3][3];   // 2D array
```

Accessing: arr[0] = 5; System.out.println(arr[0]); **Array length:** arr.length (property, not a method).

1.17 Control Statements

Decision-making:

```
if (a > b) { ... }
else if (a == b) { ... }
else { ... }

switch(day) {
    case 1: System.out.println("Monday"); break;
```

```
default: System.out.println("Invalid");
}
```

Looping:

```
for (int i = 0; i < 10; i++) { ... }

while (condition) { ... }

do { ... } while (condition);           // executes at least once

for (int x : arr) { ... }               // enhanced for-loop (for-each)
```

Jump statements: break (exit loop/switch), continue (skip to next iteration), return (exit method).

1.18 Methods and Classes

A **class** is a blueprint/template for creating objects, defining fields (data) and methods (behavior). A **method** is a block of code that performs a specific task, defined within a class.

```
class Student {
    String name;           // field
    int age;

    void display() {       // method
        System.out.println(name + " is " + age + " years old");
    }

    Student(String n, int a) { // constructor
        name = n;
        age = a;
    }

    public static void main(String[] args) {
        Student s1 = new Student("Amit", 20); // object creation
        s1.display();
    }
}
```

Method overloading: multiple methods with the **same name but different parameter lists** within the same class (compile-time/static polymorphism).

```
void add(int a, int b) { }
void add(double a, double b) { }
```

Constructor: special method used to initialize objects; same name as the class, no return type. Can be **default**, **parameterized**, or overloaded.

this keyword: refers to the current object instance (used to resolve naming conflicts between instance variables and parameters).

1.19 Inheritance

Inheritance allows a class (**subclass/child**) to acquire properties and methods of another class (**superclass/parent**), promoting code reusability. Implemented using the **extends** keyword.

```
class Animal {
    void eat() { System.out.println("This animal eats food"); }
}
class Dog extends Animal {
    void bark() { System.out.println("Dog barks"); }
}
// Dog object can use both eat() and bark()
```

Types of Inheritance in Java:

Type	Description
Single	One subclass inherits from one superclass
Multilevel	A chain: class C extends B, B extends A
Hierarchical	Multiple subclasses inherit from one superclass
Multiple	One class inherits from multiple classes — NOT supported directly for classes in Java (to avoid the “Diamond Problem”); achieved instead via interfaces

super keyword: used to refer to the immediate parent class — to call parent’s constructor (super()), or access parent’s methods/fields (super.method()).

1.20 Packages

A **package** is a namespace/mechanism that groups related classes and interfaces together, helping to organize code, avoid naming conflicts, and control access.

```
package mypackage;                // declaring a package (must be first line)

import java.util.Scanner;          // importing a built-in package/class
import mypackage.MyClass;         // importing a user-defined package
```

Types: Built-in packages (e.g., java.lang, java.util, java.io, java.awt) and **user-defined packages**.

1.21 Interfaces

An **interface** is a blueprint of a class that contains only **abstract methods** (method signatures without body) and constants (public static final fields) — (Java 8+ also allows default and static methods with bodies).

- A class **implements** an interface using the implements keyword and must provide bodies for all its abstract methods.
- Used to achieve **full abstraction** and **multiple inheritance** (a class can implement multiple interfaces, unlike extending multiple classes).

```

interface Shape {
    double area();           // abstract method (implicitly public & abstract)
}
class Circle implements Shape {
    double radius = 5;
    public double area() {
        return 3.14 * radius * radius;
    }
}

```

Class vs Interface — Key differences:

Feature	Class	Interface
Method body	Can have full implementations	Traditionally only abstract (Java 8+: default/static allowed)
Inheritance	extends (single class only)	implements (multiple interfaces allowed)
Fields	Instance/static variables	Only public static final constants
Object creation	Can be instantiated	Cannot be instantiated directly

Unit 2: Core Java — Exception Handling, Threads, I/O, Applet, AWT, JDBC

2.1 Exception Handling

An **exception** is an event that disrupts the normal flow of a program's instructions during execution (e.g., division by zero, invalid array index, file not found).

Exception hierarchy: Throwable → Exception (recoverable) and Error (serious, usually not recoverable, e.g., OutOfMemoryError). - **Checked exceptions:** checked at compile time (e.g., IOException, SQLException) — must be handled or declared. - **Unchecked exceptions:** occur at runtime (e.g., ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException).

Keywords: try, catch, finally, throw, throws

```
try {
    int result = 10 / 0;                // may throw exception
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e);
} finally {
    System.out.println("This block always executes");
}
```

- throw – used to **explicitly throw** an exception: throw new ArithmeticException("custom message");
- throws – used in a method signature to **declare** that a method might throw an exception: void readFile() throws IOException { ... }
- **Custom (user-defined) exceptions:** created by extending the Exception class.

```
class MyException extends Exception {
    MyException(String msg) { super(msg); }
}
```

Multiple catch blocks can handle different exception types for the same try block; finally always executes (used for cleanup, e.g., closing files).

2.2 Multithreaded Programming

A **thread** is the smallest unit of execution within a process; **multithreading** allows concurrent execution of two or more parts of a program to maximize CPU utilization.

Creating a thread — two ways:

1. **By extending the Thread class:**

```
class MyThread extends Thread {
    public void run() { System.out.println("Thread running"); }
}
MyThread t = new MyThread();
t.start();                // start() invokes run() in a new thread
```

2. **By implementing the Runnable interface** (preferred, since Java doesn't support multiple inheritance):

```
class MyRunnable implements Runnable {  
    public void run() { System.out.println("Thread running"); }  
}  
Thread t = new Thread(new MyRunnable());  
t.start();
```

Thread life cycle states: New → Runnable → Running → Blocked/Waiting → Terminated (Dead).

Key methods: start(), run(), sleep(ms), join() (wait for a thread to finish), yield().

Synchronization: used to control access to shared resources by multiple threads, preventing race conditions/data corruption, using the synchronized keyword on methods or blocks.

```
synchronized void updateBalance() { ... }
```

2.3 I/O (Input/Output) in Java

Java's I/O is handled through the **java.io** package, primarily using **Streams**.

- **Byte Stream:** handles raw binary data, 1 byte at a time — classes: InputStream, OutputStream (e.g., FileInputStream, FileOutputStream).
- **Character Stream:** handles character (text) data, using Unicode — classes: Reader, Writer (e.g., FileReader, FileWriter, BufferedReader).

```
import java.io.*;  
BufferedReader br = new BufferedReader(new FileReader("data.txt"));  
String line = br.readLine();  
br.close();
```

Standard I/O: System.in (input stream), System.out (output stream), System.err (error stream). Scanner class (java.util.Scanner) is commonly used for reading user input.

2.4 Applet

An **Applet** is a small Java program that runs **inside a web browser** (or applet viewer), embedded in an HTML page (Note: applets are deprecated/removed in modern Java (Java 9+) and browsers, but remain in many syllabi conceptually).

- Extends the Applet (or JApplet for Swing) class; does **not** have a main() method — execution is controlled by the browser via the **applet life cycle**.

Applet Life Cycle methods: | Method | Purpose | |—|—| | init() | Called once when the applet is first loaded — initialization | | start() | Called after init(), and every time the applet becomes visible | | paint(Graphics g) | Used to draw/render content

on the applet || stop() | Called when the applet becomes invisible/browser navigates away || destroy() | Called once when the applet is being removed from memory |

```
import java.applet.Applet;
import java.awt.Graphics;
public class MyApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello Applet", 20, 20);
    }
}
```

Embedded in HTML using: `<applet code="MyApplet.class" width="300" height="300"></applet>`

2.5 String Handling

Strings in Java are **objects** of the String class, and are **immutable** (once created, the content cannot be changed — any modification creates a new String object).

```
String s1 = "Hello";
String s2 = new String("Hello");
String s3 = s1.concat(" World");    // "Hello World" (new object)
```

Common String methods: length(), charAt(i), substring(start,end), equals(), equalsIgnoreCase(), compareTo(), toUpperCase(), toLowerCase(), trim(), replace(), split(), indexOf(), concat().

StringBuffer / StringBuilder: **mutable** sequences of characters (unlike String), used when frequent modifications are needed. StringBuffer is **thread-safe (synchronized)**; StringBuilder is **faster but not thread-safe** (Java 5+).

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");    // modifies same object, no new object created
```

2.6 Networking

Java provides networking support through the **java.net** package, enabling communication between programs over a network.

Class	Purpose
InetAddress	Represents an IP address
Socket	Enables client-side TCP connection to a server
ServerSocket	Enables server to listen for and accept client connections
DatagramSocket / DatagramPacket	Used for connectionless (UDP) communication
URL / URLConnection	Represents and connects to a web resource

Simple Client-Server (TCP) example (concept):

```
// Server side
ServerSocket server = new ServerSocket(5000);
Socket socket = server.accept();           // waits for client connection

// Client side
Socket socket = new Socket("localhost", 5000);
```

2.7 Event Handling

Java uses the **Delegation Event Model** — a **source** (e.g., a button) generates an **event**, which is handled by a registered **listener** object.

Steps: 1) Implement a suitable Listener interface. 2) Override its callback method(s). 3) Register the listener with the source component using an `addXListener()` method.

```
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }
});
```

Common event classes/listeners: | Event Class | Listener Interface | Triggered by |
|—|—|—| | ActionEvent | ActionListener | Button click, menu selection | | MouseEvent |
MouseListener / MouseMotionListener | Mouse click, move, drag | | KeyEvent |
KeyListener | Keyboard key press/release | | WindowEvent | WindowListener | Window
open/close/minimize | | ItemEvent | ItemListener | Checkbox/list item selection |

2.8 Introduction to AWT (Abstract Window Toolkit)

AWT (java.awt package) is Java's original **platform-dependent** GUI toolkit for building graphical user interfaces — components are rendered using the native OS's GUI elements ("heavyweight" components).

Core AWT hierarchy: Component → Container → Window/Panel → Frame, Dialog, etc.

```
import java.awt.*;
class MyFrame extends Frame {
    MyFrame() {
        setSize(300,200);
        setTitle("AWT Frame");
        setVisible(true);
    }
    public static void main(String[] a) { new MyFrame(); }
}
```

2.9 AWT Controls (Components)

Control	Class	Purpose
Label	Label	Displays non-editable text
Button	Button	Clickable push button
TextField	TextField	Single-line text input
TextArea	TextArea	Multi-line text input
Checkbox	Checkbox	Toggle on/off option
CheckboxGroup	CheckboxGroup	Groups checkboxes into radio-button behavior
Choice	Choice	Drop-down list (single selection)
List	List	Scrollable list (single/multiple selection)
Scrollbar	Scrollbar	Horizontal/vertical scroll control

2.10 Layout Managers

Control **how components are arranged** within a container (instead of fixed pixel positioning).

Layout Manager	Arrangement
FlowLayout	Components placed left-to-right, wrapping to next line (default for Panel)
BorderLayout	Divides container into 5 regions: North, South, East, West, Center (default for Frame)
GridLayout	Arranges components in a rectangular grid of rows/columns
CardLayout	Stacks components like a deck of cards; only one visible at a time
GridBagLayout	Flexible grid; components can span multiple rows/columns with fine control

2.11 Menus

Menus provide drop-down command options in a window, built using MenuBar, Menu, and MenuItem.

```
MenuBar mb = new MenuBar();
Menu fileMenu = new Menu("File");
MenuItem open = new MenuItem("Open");
fileMenu.add(open);
mb.add(fileMenu);
frame.setMenuBar(mb);
```

2.12 Images and Graphics

- **Graphics class (java.awt.Graphics):** provides methods to draw shapes/text/images on a component, e.g., drawLine(), drawRect(), fillRect(), drawOval(), drawString(), drawImage().
- **Image handling:** images loaded using Toolkit.getDefaultToolkit().getImage() or ImageIO.read(), then rendered using g.drawImage(img, x, y, this).

```
public void paint(Graphics g) {  
    g.setColor(Color.RED);  
    g.fillOval(50, 50, 100, 100);  
    g.drawString("Circle", 60, 160);  
}
```

2.13 JDBC (Java Database Connectivity)

JDBC (java.sql package) is an API that enables Java programs to connect to and interact with relational databases (execute SQL queries, retrieve/update data).

Steps to connect using JDBC: 1. **Load/Register the driver:** Class.forName("com.mysql.cj.jdbc.Driver"); (implicit in modern JDBC 4+) 2. **Establish connection:** Connection con = DriverManager.getConnection(url, user, password); 3. **Create a Statement:** Statement stmt = con.createStatement(); 4. **Execute a query:** ResultSet rs = stmt.executeQuery("SELECT * FROM students"); 5. **Process results:** iterate using rs.next(), retrieve columns using rs.getString(), rs.getInt(), etc. 6. **Close the connection:** con.close();

```
import java.sql.*;  
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/mydb", "root", "password");  
Statement stmt = con.createStatement();  
ResultSet rs = stmt.executeQuery("SELECT * FROM students");  
while (rs.next()) {  
    System.out.println(rs.getString("name"));  
}  
con.close();
```

JDBC Driver Types: | Type | Description | |—|—| | Type 1 (JDBC-ODBC Bridge) | Translates JDBC calls into ODBC calls (obsolete) | | Type 2 (Native-API driver) | Uses database's native client library | | Type 3 (Network Protocol driver) | Uses a middle-ware server to translate calls | | Type 4 (Thin/Pure Java driver) | Directly converts JDBC calls into database-specific protocol (most widely used today) |

Unit 3: Java Servlets and JSP

Part A: Java Servlets

3.1 Introduction to Servlets

A **Servlet** is a Java program that runs on a **web server** and extends the server's capability to handle client requests and generate dynamic responses (typically HTML), following the **request-response model** over HTTP.

- Servlets execute inside a **Servlet Container** (e.g., Apache Tomcat), which manages their lifecycle.
- Belongs to the `javax.servlet` and `javax.servlet.http` packages.
- **Advantages over CGI:** better performance (each request handled by a lightweight thread, not a new process), platform-independent, robust, secure.

3.2 Servlet Life Cycle

Managed by the Servlet Container, consisting of these phases:

Phase	Method	Description
1. Loading & Instantiation	(container loads the class)	Servlet class is loaded and an instance created
2. Initialization	<code>init()</code>	Called once , when the servlet is first loaded — used for one-time setup (e.g., DB connection)
3. Request Handling	<code>service()</code>	Called for every client request ; dispatches to <code>doGet()/doPost()</code> based on HTTP method
4. Destruction	<code>destroy()</code>	Called once , before the servlet is removed from service — used for cleanup

`init()` → `service()` [called repeatedly for each request] → `destroy()`

```
public class MyServlet extends HttpServlet {
    public void init() { System.out.println("Servlet initialized"); }
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h1>Hello from Servlet</h1>");
    }
    public void destroy() { System.out.println("Servlet destroyed"); }
}
```

3.3 HTTP Servlet Class

HttpServlet (abstract class in `javax.servlet.http`) is the most commonly extended class for creating servlets that handle **HTTP-specific** requests. It overrides `service()` internally to dispatch requests to specific handler methods based on HTTP method type:

Method	Handles HTTP
<code>doGet()</code>	GET requests (e.g., fetching a page, query-string data)
<code>doPost()</code>	POST requests (e.g., form submissions with body data)
<code>doPut()</code>	PUT requests
<code>doDelete()</code>	DELETE requests
<code>doHead()</code>	HEAD requests

3.4 Request Interface (`HttpServletRequest`)

Represents the **client's request**; used to retrieve data sent by the client (form parameters, headers, cookies, session).

Common methods: | Method | Purpose | |—|—| | `getParameter(String name)` | Returns the value of a request parameter (form field) | | `getParameterNames()` | Returns all parameter names | | `getHeader(String name)` | Returns value of a specific HTTP header | | `getMethod()` | Returns HTTP method used (GET/POST) | | `getSession()` | Returns the current `HttpSession` object (creates one if none exists) | | `getCookies()` | Returns array of cookies sent by the client | | `getRequestURI()` | Returns the URL path of the request |

3.5 Response Interface (`HttpServletResponse`)

Represents the **server's response** sent back to the client; used to construct and send output.

Common methods: | Method | Purpose | |—|—| | `setContentType(String type)` | Sets the MIME type of the response (e.g., "text/html") | | `getWriter()` | Returns a `PrintWriter` to write character-based response content | | `getOutputStream()` | Returns a stream to write binary response content | | `addCookie(Cookie c)` | Adds a cookie to the response | | `sendRedirect(String url)` | Redirects the client to a different URL | | `setStatus(int sc)` | Sets the HTTP status code of the response |

3.6 Session Tracking

HTTP is a **stateless** protocol — each request is independent, and the server does not automatically remember previous requests from the same client. **Session tracking** techniques are used to maintain state (user identity/data) across multiple requests.

Techniques:

Technique	Description
Cookies	Small pieces of data sent by the server and stored on the client's browser, automatically resent with every subsequent request to the same server. Created using Cookie class: <code>Cookie c = new Cookie("user", "Amit"); response.addCookie(c);</code>
URL Rewriting	Session information is appended directly to the URL as extra parameters (e.g., <code>page.jsp?sessionid=123</code>) — useful when cookies are disabled by the client
Hidden Form Fields	Session data stored in hidden <code><input type="hidden"></code> fields within a form, resent on each form submission
HttpSession Object	Server-side session object (<code>request.getSession()</code>) that stores user-specific data on the server, identified by a unique session ID (usually transmitted via a cookie, <code>JSESSIONID</code>) — the most robust and commonly used method

```
HttpSession session = request.getSession();
session.setAttribute("username", "Amit");
String user = (String) session.getAttribute("username");
```

Part B: JSP (JavaServer Pages)

3.7 Overview of JSP

JSP (JavaServer Pages) is a technology used to create dynamic web content by **embedding Java code directly within HTML pages**, using special tags. When a JSP page is requested for the first time, the server **translates it into a servlet**, compiles, and executes it — subsequent requests reuse the compiled servlet (faster).

Advantages of JSP over plain Servlets: easier to write/maintain (HTML-centric rather than Java-centric with embedded `out.println()` calls), separates presentation from business logic, supports custom tags.

3.8 Relation of Applet and Servlet with JSP

Technology	Executes on	Purpose
Applet	Client-side (inside browser)	Provides rich client-side GUI/interactivity

Technology	Executes on	Purpose
Servlet	Server-side	Handles requests and generates dynamic content; JSP is internally converted into a servlet by the container
JSP	Server-side (compiled to servlet)	Simplifies dynamic web page creation by mixing HTML and Java code directly

Essentially, **JSP = HTML + Servlet**, offering an easier way to write servlet-like functionality, while a servlet handling GUI logic on the client corresponds conceptually to what an Applet would do on the client side.

3.9 JSP Scripting Elements

JSP provides three types of scripting elements to embed Java code within an HTML page:

1. JSP Expression `<%= expression %>` — evaluates a Java expression and inserts the result (as a String) directly into the output.

```
<%= "Current time: " + new java.util.Date() %>
```

2. JSP Scriptlet `<% code %>` — contains Java code (statements) executed each time the page is requested; can contain loops, conditions, variable declarations.

```
<% for (int i = 1; i <= 5; i++) { %>
    Number: <%= i %><br>
<% } %>
```

3. JSP Declaration `<%! declaration %>` — declares methods or variables (at class level) that become members of the generated servlet class (not inside `_jspService()`).

```
<%! int counter = 0;
    int increment() { return ++counter; } %>
```

JSP Directives (`<%@ ... %>`) — provide global instructions to the container about the page (not directly output): - `<%@ page ... %>` — page-level settings (import packages, error page, content type) - `<%@ include file="header.jsp" %>` — includes another file's content at translation time - `<%@ taglib ... %>` — declares a custom tag library used in the page

3.10 Predefined (Implicit) Objects in JSP

JSP automatically provides these objects without explicit declaration:

Object	Type	Purpose
request	HttpServletRequest	Client's request data
response	HttpServletResponse	Server's response
out	JspWriter	Used to write output to the client
session	HttpSession	Session data for the current user
application	ServletContext	Shared data across the whole application
config	ServletConfig	Servlet configuration information
pageContext	PageContext	Access to various namespaces of the JSP page
page	Object	Reference to the current JSP page instance (this)
exception	Throwable	Available only in error pages; holds the exception object

3.11 Creating Custom JSP Tag Libraries

JSP allows developers to create their own **custom tags** (instead of using only scripting elements), improving code reuse and cleanly separating logic from presentation.

Steps to create a custom tag: 1. Create a **Tag Handler class** implementing Tag/SimpleTag interface (or extending TagSupport/SimpleTagSupport), defining the tag's behavior. 2. Create a **Tag Library Descriptor (TLD) file** — an XML file that maps tag names to their handler classes. 3. **Declare the tag library** in the JSP page using the taglib directive: `<%@ taglib uri="mytags.tld" prefix="mt" %>` 4. **Use the custom tag** in the JSP page: `<mt:greet name="Amit"/>`

3.12 Using Nested Tags

Custom tags can be **nested** inside one another, allowing a parent tag to control/process the content and behavior of child tags (e.g., a looping tag that repeatedly processes its nested body content). The tag handler can access its parent tag using methods like `getParent()`, enabling communication and coordinated behavior between nested tags.

3.13 Structuring the Generated Servlet in JSP Pages

When a JSP page is translated by the container, its various pieces map into the resulting servlet as follows: - **Static HTML content** → converted into `out.print()` statements. - **Scriptlets** (`<% %>`) → inserted directly into the servlet's `_jspService()` method. - **Expressions** (`<%= %>`) → converted into `out.print(expression)`. - **Declarations** (`<%! %>`) → become member variables/methods of the generated servlet

class (outside `_jspService()`). - **Directives** → affect servlet-level settings (imports, superclass, etc.), not converted into executable statements.

3.14 Including Files and Applets in JSP Documents

Including files: - **Static include** (`<%@ include file="..." %>`) — content is merged **at translation/compile time** (before the JSP is compiled into a servlet); faster, but changes to the included file require recompilation. - **Dynamic include** (`<jsp:include page="..." />`) — content is included **at request time**; more flexible, slightly slower.

Including an Applet in JSP: Simply embed standard HTML `<applet>` tags within the JSP page (since JSP ultimately renders as HTML sent to the browser); the applet then runs client-side as usual once the page loads.

3.15 Integrating Servlet and JSP

Servlets and JSP are often used together following the **MVC (Model-View-Controller)** design pattern: - **Servlet (Controller):** handles incoming requests, executes business logic, interacts with the Model (e.g., database). - **JSP (View):** responsible only for presentation — displaying data passed to it. - **JavaBean/Model:** represents the data/business objects.

Request Dispatching between Servlet and JSP:

```
RequestDispatcher rd = request.getRequestDispatcher("result.jsp");
request.setAttribute("data", someValue);
rd.forward(request, response);    // forwards control to JSP, server-side
```

This combination lets Servlets handle logic/control flow while JSP handles clean presentation, resulting in better-organized, maintainable web applications.

Unit 4: Java Swing

4.1 Introduction to Swing

Swing (javax.swing package) is a GUI toolkit built on top of AWT that provides a richer set of **lightweight** (pure Java, not OS-dependent) components, with a **pluggable look and feel**, more features, and better performance than AWT.

Swing vs AWT: | Feature | AWT | Swing | |—|—|—| | Component type | Heavyweight (uses native OS components) | Lightweight (rendered entirely in Java) | | Look and feel | Fixed (OS-dependent) | Pluggable (can change appearance) | | Components | Basic (Button, Label, TextField) | Rich (JTable, JTree, JTabbedPane, etc.) | | Package | java.awt | javax.swing | | MVC support | No | Yes (Swing components follow MVC internally) |

Swing component names are generally prefixed with “J” (e.g., JButton, JLabel, JFrame), distinguishing them from their AWT counterparts.

4.2 Creating a Swing Applet and Application

Swing Application (standalone, using JFrame):

```
import javax.swing.*;
public class MyApp {
    public static void main(String[] args) {
        JFrame frame = new JFrame("My Swing App");
        JButton btn = new JButton("Click Me");
        frame.add(btn);
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Swing Applet (using JApplet — deprecated in modern Java, but conceptually):

```
import javax.swing.*;
public class MyJApplet extends JApplet {
    public void init() {
        JButton btn = new JButton("Swing Applet Button");
        getContentPane().add(btn);
    }
}
```

4.3 Programming Using Panes

Swing’s top-level containers (JFrame, JApplet, JDialog) use a layered structure of “panes”:

Pane	Purpose
RootPane	Top-level pane managing the other panes
ContentPane	The main container where visible components (buttons, labels) are added: <code>frame.getContentPane().add(component);</code>
GlassPane	Transparent pane on top of everything — used for capturing input events or drawing over all components
LayeredPane	Manages the Z-order (depth/stacking) of components (see 4.13)

4.4 Pluggable Look and Feel (PLAF)

Swing allows the visual appearance (look and feel) of the entire application to be changed at runtime, independent of the underlying platform.

```
UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
```

Common look-and-feels: **Metal** (default, cross-platform), **Nimbus**, **Motif**, **Windows/System** look and feel.

4.5 Basic Swing Controls

Component	Class	Purpose
Label	JLabel	Displays non-editable text/icon
Text Field	JTextField	Single-line editable text input
Text Area	JTextArea	Multi-line editable text
Button	JButton	Standard clickable push button
Toggle Button	JToggleButton	Button with two states (pressed/not pressed), stays pressed until clicked again
Checkbox	JCheckBox	Independent on/off selection option
Radio Button	JRadioButton (grouped via ButtonGroup)	Mutually exclusive selection among a group

```
JLabel label = new JLabel("Name:");
JTextField tf = new JTextField(15);
JButton btn = new JButton("Submit");
JCheckBox cb = new JCheckBox("Subscribe");
JRadioButton rl = new JRadioButton("Male");
```

```
JRadioButton r2 = new JRadioButton("Female");
ButtonGroup bg = new ButtonGroup();
bg.add(r1); bg.add(r2);
```

4.6 Viewport, Scroll Panes, and Scroll Bars

- **JViewport:** provides a window through which a larger component can be viewed — the scrollable “peephole” mechanism underlying JScrollPane.
- **JScrollPane:** adds automatic horizontal/vertical scrollbars around a component (e.g., a large JTextArea or JTable) that doesn’t fit in the visible area.

```
JTextArea ta = new JTextArea(20, 40);
JScrollPane sp = new JScrollPane(ta);
frame.add(sp);
```

- **JScrollBar:** a standalone scroll bar control that can be used independently to select a value from a range.

4.7 Lists and Combo Box

- **JList:** displays a list of items, allows single or multiple selection.

```
String[] items = {"Red", "Green", "Blue"};
JList<String> list = new JList<>(items);
```

- **JComboBox:** a drop-down list allowing selection of a single item from a list (combines a text field and a drop-down list); saves screen space compared to JList.

```
JComboBox<String> combo = new JComboBox<>(items);
```

4.8 Progress Bar

JProgressBar visually displays the progress of a lengthy task/operation as a filled bar, typically ranging from a minimum to a maximum value.

```
JProgressBar pb = new JProgressBar(0, 100);
pb.setValue(50);
pb.setStringPainted(true); // shows percentage text
```

4.9 Menus and Toolbars

- **Menus:** built using JMenuBar, JMenu, JMenuItem (similar structure to AWT menus but lightweight and richer, supporting icons, checkbox/radio menu items, keyboard mnemonics/accelerators).

```
JMenuBar mb = new JMenuBar();
JMenu fileMenu = new JMenu("File");
JMenuItem openItem = new JMenuItem("Open");
fileMenu.add(openItem);
```

```
mb.add(fileMenu);
frame.setJMenuBar(mb);
```

- **JToolBar**: a movable/floatable bar containing buttons/controls for frequently used commands (shortcuts to menu actions), can be dragged to different edges of the window or made to float separately.

4.10 Layered Panes

JLayeredPane manages components in **overlapping layers** (Z-order/depth), allowing components to be stacked on top of one another (e.g., displaying a tooltip or popup above other components) — components added to a higher layer number appear on top of those in lower layers.

4.11 Tabbed Panes

JTabbedPane organizes multiple components into a group of “tabs,” where only the component/panel associated with the currently selected tab is visible — used to save space by presenting multiple views/pages within a single window.

```
JTabbedPane tp = new JTabbedPane();
tp.addTab("Tab1", panel1);
tp.addTab("Tab2", panel2);
```

4.12 Split Panes

JSplitPane divides a container into **two resizable sections** (either side-by-side or top-bottom), separated by a draggable divider, allowing the user to adjust the relative space given to each section.

```
JSplitPane split = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT, leftPanel, rightPanel);
```

4.13 Layouts (Swing Layout Managers)

Swing supports all AWT layout managers (FlowLayout, BorderLayout, GridLayout, CardLayout, GridBagLayout) plus additional ones: | Layout | Description | |—|—| | **BoxLayout** | Arranges components in a single row or column, respecting preferred sizes | | **SpringLayout** | Flexible layout defined by specifying spring-like constraints between component edges | | **GroupLayout** | Used heavily by GUI builders (like NetBeans); groups components sequentially/parallel with fine alignment control |

4.14 Windows and Dialog Boxes

- **JWindow**: a top-level window with **no title bar, borders, or menu bar** (rarely used directly; useful for splash screens).
- **JDialog**: a secondary top-level window (typically with a title bar and border) used to take input or show messages, often **modal** (blocks interaction with other

windows until closed) — commonly used via convenience class `JOptionPane` for standard message/input/confirmation dialogs.

```
JOptionPane.showMessageDialog(frame, "Operation Successful!");  
int choice = JOptionPane.showConfirmDialog(frame, "Are you sure?");
```

4.15 Internal Frame

JInternalFrame is a lightweight “window” that exists **within another container** (typically a `JDesktopPane`), used to implement **MDI (Multiple Document Interface)** applications, where multiple internal frames/windows can be opened, moved, resized, minimized, and closed **within a single main application window** (rather than as separate OS-level windows).

```
JDesktopPane desktop = new JDesktopPane();  
JInternalFrame iframe = new JInternalFrame("Document 1", true, true, true, true);  
iframe.setSize(200, 150);  
iframe.setVisible(true);  
desktop.add(iframe);
```

Unit 5: Java Beans, Enterprise Java Beans (EJB), and RMI

5.1 Introduction to Java Beans

A **JavaBean** is a reusable, self-contained Java software component that can be manipulated visually in a builder/IDE tool. It follows a specific set of conventions so that tools can automatically discover its properties, methods, and events.

JavaBean conventions: - Must be a **public class** with a **public no-argument (default) constructor**. - Properties should be **private**, accessed via **public getter/setter methods** following naming conventions: `getPropertyName()` / `setPropertyName(value)` (for boolean properties, `isPropertyName()`). - Should be **Serializable** (implements `java.io.Serializable`) so its state can be saved/restored.

```
import java.io.Serializable;
public class StudentBean implements Serializable {
    private String name;
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

5.2 Application Builder Tools

Builder tools (visual IDEs, e.g., NetBeans, older tools like Sun's Bean Box) allow developers to **visually assemble applications** by dragging and dropping JavaBean components onto a design surface, setting their properties through property sheets, and connecting their events to other components — without manually writing all the boilerplate code.

5.3 The Bean Developer Kit (BDK)

The **BDK** (from Sun Microsystems) was a toolkit that provided the **BeanBox** — a container/test environment for visually testing and experimenting with JavaBeans (viewing their properties, testing event hookups) before integrating them into a full application. It served as a reference implementation demonstrating the JavaBeans component model.

5.4 JAR Files

A **JAR (Java Archive)** file packages multiple `.class` files (and related resources like images, and a manifest file) into a single **compressed** file, similar to a ZIP archive — used to distribute and deploy Java Beans/applications/libraries as a single unit.

Creating a JAR: `jar cf mybean.jar MyBean.class` **Manifest file:** a special file (`META-INF/MANIFEST.MF`) inside the JAR that can specify which classes are Beans, the main class to run, and other metadata (e.g., `Java-Bean: True`).

5.5 Introspection

Introspection is the process by which a builder tool **automatically analyzes a Bean** to discover its properties, methods, and events, using **Java Reflection** and following naming/design pattern conventions (getters/setters, addXListener/removeXListener methods).

- A Bean developer can also provide an explicit **BeanInfo** class (implementing BeanInfo interface) to give the builder tool more detailed/customized information about the Bean than what automatic introspection would discover.

5.6 Developing a Simple Bean

Example — a simple Bean with a property:

```
import java.io.Serializable;

public class ColorBean implements Serializable {
    private String color = "White";

    public String getColor() {
        return color;
    }
    public void setColor(String color) {
        String old = this.color;
        this.color = color;
    }
}
```

This class satisfies the JavaBean conventions: public no-arg constructor (implicit default), a private property color, and standard getter/setter methods — this makes it recognizable and usable by any Bean-aware builder tool.

5.7 Using Bound Properties

A **bound property** is a Bean property that **notifies other objects (listeners) whenever its value changes**, using the **Observer pattern** via PropertyChangeSupport and PropertyChangeListener.

```
import java.beans.PropertyChangeSupport;
import java.beans.PropertyChangeListener;

public class TemperatureBean {
    private double temp;
    private PropertyChangeSupport support = new PropertyChangeSupport(this);

    public void addPropertyChangeListener(PropertyChangeListener l) {
        support.addPropertyChangeListener(l);
    }
    public void setTemp(double newTemp) {
```

```

        double old = temp;
        temp = newTemp;
        support.firePropertyChange("temp", old, newTemp); // notify listeners
    }
}

```

Used when a change in one Bean's property should automatically trigger updates in other connected Beans/components (e.g., updating a display label whenever a data value changes).

5.8 The Java Bean API

The JavaBeans API is provided mainly through the **java.beans** package, offering classes/interfaces to support introspection, event handling, and customization:

Class/Interface	Purpose
Introspector	Analyzes a Bean class and produces a BeanInfo
BeanInfo	Provides explicit information about a Bean's properties/methods/events
PropertyDescriptor	Describes a single property of a Bean
EventSetDescriptor	Describes an event that a Bean can fire
MethodDescriptor	Describes a method of a Bean
PropertyChangeSupport / PropertyChangeListener	Support for bound properties
Customizer	Allows a custom GUI for editing a Bean's properties in a builder tool

5.9 Session Beans (EJB)

A **Session Bean** is a type of Enterprise Java Bean that represents a **business process/task** for a single client, and does **not represent persistent data** (its state is not stored in a database) — it exists only for the duration of a client's interaction/session.

Type	Description
Stateless Session Bean	Does not maintain any conversational state between method calls; any instance can serve any client — highly scalable
Stateful Session Bean	Maintains conversational state specific to a particular client across multiple method calls, for the duration of a session

5.10 Entity Beans (EJB)

An **Entity Bean** represents **persistent business data** (typically corresponding to a row in a database table) — its state is stored permanently and survives beyond a single client session/server restart.

Type	Description
Container-Managed Persistence (CMP)	The EJB container automatically handles all database interaction (save/load) for the entity bean
Bean-Managed Persistence (BMP)	The Bean's own code explicitly handles database interaction (using JDBC calls written by the developer)

(Note: in modern Java EE/Jakarta EE, Entity Beans have largely been replaced by JPA — Java Persistence API — but Entity Beans remain a key historical/conceptual EJB topic.)

5.11 Introduction to Enterprise Java Beans (EJB)

EJB is a **server-side component architecture** for building scalable, secure, and transactional distributed business applications, deployed and managed within an **EJB container** (part of a Java EE application server).

Key benefits provided automatically by the EJB container: - **Transaction management** (automatic commit/rollback) - **Security** (authentication/authorization) - **Concurrency and lifecycle management** (pooling and management of Bean instances) - **Remote access** (clients can invoke EJB methods remotely, typically via RMI)

Types of EJBs: **Session Beans** (business logic), **Entity Beans** (persistent data), and **Message-Driven Beans** (process asynchronous messages, e.g., from a JMS queue).

5.12 Introduction to RMI (Remote Method Invocation)

RMI is a Java API that allows an object running in **one JVM** to invoke methods on an object running in **another JVM** (possibly on a different physical machine), enabling distributed applications where objects communicate as if they were local — the underlying network communication is handled transparently by RMI.

Key RMI Components: | **Component** | **Role** | |—|—| | **Remote Interface** | Declares the methods that can be called remotely; must extend `java.rmi.Remote`, and each method must declare `throws RemoteException` | | **Remote Object (Implementation)** | The class that implements the remote interface, typically extending `UnicastRemoteObject` | | **Stub** | Client-side proxy object; represents the remote object and forwards method calls to it over the network | | **Skeleton** | Server-side helper (mostly obsolete in modern RMI) that used to receive calls and forward them to the actual remote object | | **RMI Registry** | A naming service (`rmiregistry`) where remote objects are registered with a name, allowing clients to look them up |

5.12.1 Steps to Build a Simple RMI Client-Server Application

1. Define the Remote Interface:

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

2. Implement the Remote Interface (Server object):

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    protected CalculatorImpl() throws RemoteException { super(); }
    public int add(int a, int b) throws RemoteException {
        return a + b;
    }
}
```

3. Create the Server (registers the remote object):

```
import java.rmi.registry.LocateRegistry;
import java.rmi.Naming;

public class Server {
    public static void main(String[] args) throws Exception {
        LocateRegistry.createRegistry(1099);
        Calculator calc = new CalculatorImpl();
        Naming.rebind("rmi://localhost/CalcService", calc);
        System.out.println("Server ready...");
    }
}
```

4. Create the Client (looks up and calls the remote object):

```
import java.rmi.Naming;

public class Client {
    public static void main(String[] args) throws Exception {
        Calculator calc = (Calculator) Naming.lookup("rmi://localhost/CalcService");
        int result = calc.add(5, 10);
        System.out.println("Result: " + result);
    }
}
```

Execution steps (exam point): 1. Compile all files (javac *.java). 2. Start the RMI registry (rmiregistry or via LocateRegistry.createRegistry() in code). 3. Run

the Server (registers the remote object under a name). 4. Run the Client (looks up the object by name and invokes its remote methods as if it were local).